

手撕一个Deep Research项目

讲师：肖斌

第一章：Deep Research相关概念

Deep Research（深度研究） 是当前AI领域最前沿的能力之一，它标志着大语言模型从“被动回答者”向“主动研究者”的范式转变。其核心是赋予AI一个端到端的自动化研究 workflows，使其能够像人类研究员一样，对复杂、开放的查询进行规划、搜索、分析、整合，最终生成一份有据可查、结构化的深度报告。

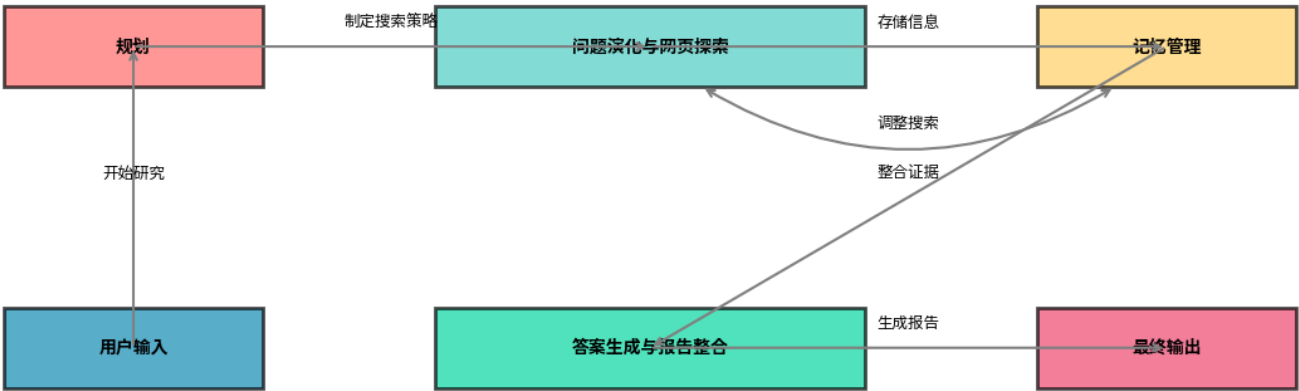


一、核心理念：从“回答问题”到“进行研究”

传统AI问答或基础的RAG（检索增强生成）是**被动响应**：用户提问，模型基于已有知识或一次检索的结果生成答案。而Deep Research是**主动探索**：它将一个复杂问题视为一个研究项目，自主规划研究路径，进行多轮、迭代式的信息搜集与验证，直至形成可靠结论。

二、技术架构：四大核心组件

研究循环：规划→检索→记忆→整合→输出



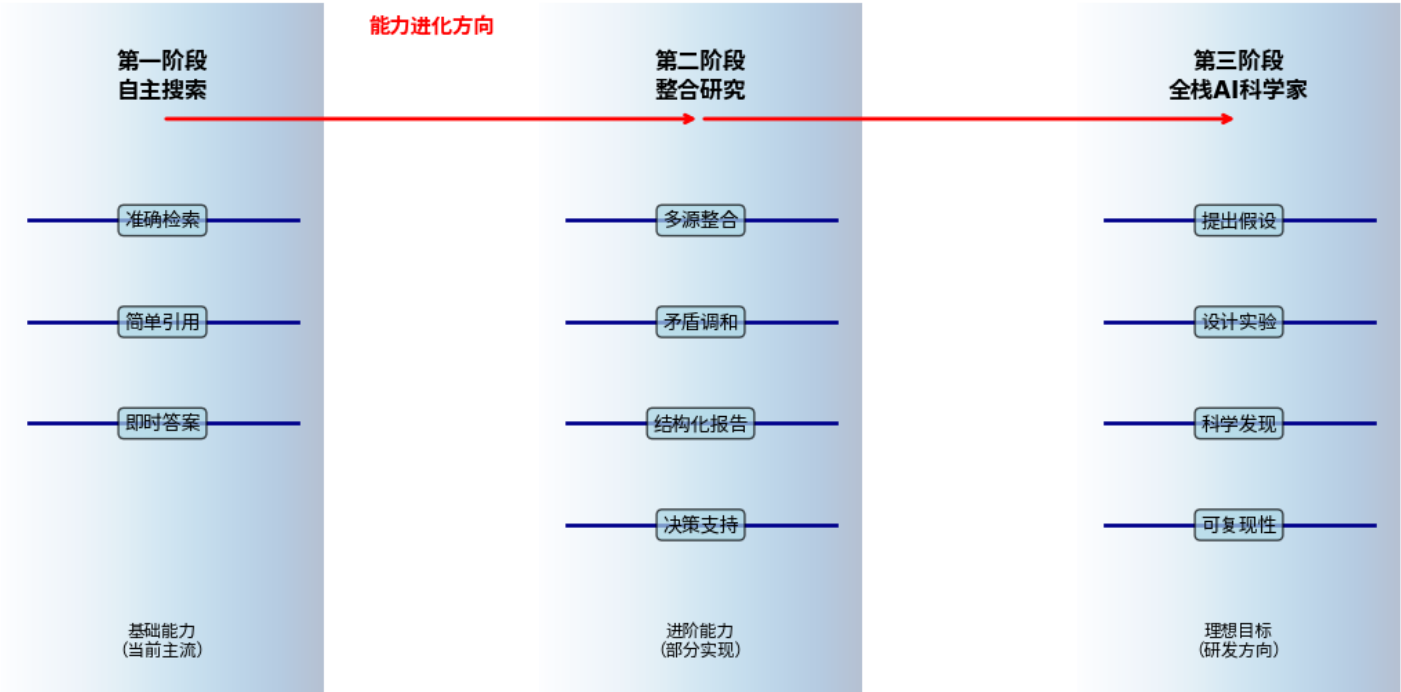
一个典型的Deep Research系统通常包含以下闭环工作流：

1. **规划**：将用户的模糊需求或复杂问题，分解为一系列可执行的子任务和研究步骤。这决定了研究的深度和方向。
2. **问题演化与网页探索**：基于规划，系统会动态生成搜索查询，主动在互联网（或指定数据库）中进行多轮、深入的检索。它不仅收集信息，还会评估信息是否足够、识别知识缺口，并调整搜索策略以填补缺口。
3. **记忆管理**：在长周期、多步骤的研究过程中，系统需要有效地存储、索引、更新和调用已获取的信息片段，以保持推理的连贯性和上下文相关性。
4. **答案生成与报告整合**：最后，系统将所有经过验证的多源证据进行综合分析，调和可能的矛盾，并生成一份结构完整、引用清晰、逻辑连贯的最终报告或答案。

三、发展历程与能力阶段

研究界将Deep Research的能力演进划分为三个阶段：

- **第一阶段：自主搜索**：专注于准确、高效地找到正确答案所需的信息源，输出简洁、有引用的答案。
- **第二阶段：整合研究**：能够将多源证据综合成连贯、结构化的长篇报告，处理信息中的冲突和不确定性，支持决策。
- **第三阶段：全栈AI科学家（理想目标）**：不仅能整合信息，还能主动提出假设、设计验证方法、批判现有观点，追求科学发现的新颖性和可复现性。



四、与相关技术的区别

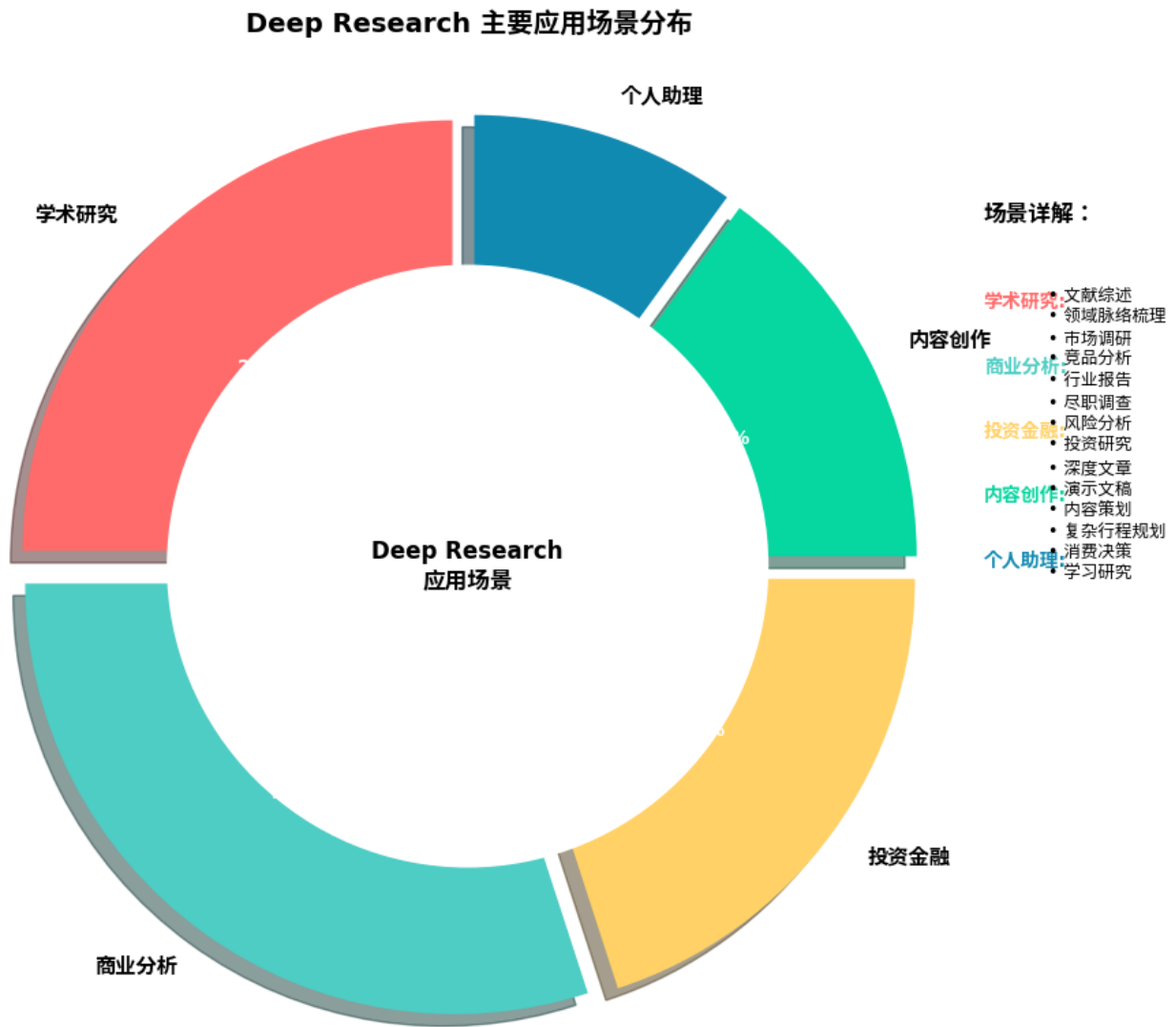
- **与传统搜索：**传统搜索返回链接列表，需要用户自行筛选、阅读和整合。Deep Research直接交付整合后的结论和依据。
- **与基础RAG：**传统RAG是被动地利用一次检索的结果来增强生成，缺乏主动规划和多轮深入探索的能力。Deep Research在此基础上增加了“思考-行动”的循环，是RAG的进化形态。

对比维度	传统AI问答 / 基础RAG	Deep Research（深度研究）
工作模式	被动响应式	主动探索式
信息处理	单次检索，直接生成	多轮迭代，规划-检索-分析循环
输出目标	给出“答案”	交付“研究成果”
信息验证	有限验证或直接信任	主动验证、交叉比对、矛盾调和
适用场景	事实性问答、简单分析	复杂问题研究、市场分析、学术综述

五、主要应用场景

Deep Research正在多个对信息深度和准确性要求高的领域发挥价值：

- **学术研究：**快速撰写文献综述、梳理领域发展脉络。
- **商业分析：**进行市场调研、竞品分析、行业趋势报告生成。
- **投资与金融：**自动化完成早期的尽职调查信息收集，整合市场、竞争和风险信息。
- **内容创作：**辅助生成深度文章、演示文稿（PPT）等。
- **个人助理：**规划复杂行程（如结合赛事、交通、娱乐的旅行）、进行深度消费决策调研等。



六、业界主要实践者

- **谷歌 Gemini Deep Research**：基于Gemini 3 Pro模型，强调低幻觉率和高事实性。它已深度集成 Google Workspace，可以调用用户的Gmail、Drive文档等私人数据来生成个性化报告。谷歌还开源了评估基准DeepSearchQA。
- **OpenAI**：虽未以完全相同的名称推出，但其通过GPT系列模型和联网搜索等功能组合，提供了类似的深度研究能力。其o3-deep-research等模型是相关技术的体现。
- **阿里通义DeepResearch**：阿里已将整套技术开源，其模型在多个权威评测中达到领先水平，并已应用于高德地图、通义法睿等产品中。
- **百度千帆Deep Research**：作为企业级解决方案推出，依托其Agent Infra，旨在为企业提供专业级的研究报告生成能力。
- **微软Azure AI Foundry**：提供了Deep Research的API和SDK预览版，方便开发者将深度研究能力构建到自己的企业应用和工作流中。

第二章：采用DeepAgents框架来手撕开发

一、核心工具概览

DeepAgents框架通过两个核心内置工具实现了智能体的高级规划与协作能力：

工具名称	类型	主要功能	在Deep Research案例中的
write_todos	规划工具	将复杂任务分解为结构化待办事项列表	研究任务分解、进度跟踪、防止注意力
task	委派工具	将子任务委派给子智能体执行	上下文隔离、专业化分工、并行处理

二、write_todos：智能规划引擎

1. 核心理念

`write_todos` 是DeepAgents框架中最精妙的设计之一，本质上是一个**"空操作"(no-op)工具**。它的主要目的不是执行外部动作，而是通过**"语境工程"(Context Engineering)**强迫智能体将思考过程外化为结构化的任务清单，并固定在上下文中，从而对抗**"注意力漂移"**。

`write_todos` 工具是 Deep Agents 框架中 `TodoListMiddleware` 的核心组件，旨在帮助智能体对复杂的多步骤任务进行规划、拆解和动态调整。

当智能体面对大型或复杂的任务时，该工具允许它将任务分解为一系列离散的步骤，从而能够跟踪进度并根据执行过程中的新信息实时更新计划。

2. 主要功能与工作机制

- 任务拆解： 在开始执行多部分任务之前，智能体被预设为通过 `write_todos` 工具创建待办事项清单，明确当前任务的路径。
- 进度跟踪： 智能体利用该工具记录已经完成的工作和仍需执行的后续步骤，保持对任务状态的清晰管理。
- 动态调整： 该工具支持在任务执行过程中修改待办列表，使智能体能够根据任务执行情况或环境反馈灵活调整后续行动。

关键特性：

- 显式任务建模： 将自然语言任务转为结构化数据
- 可追踪状态： 每个任务具备生命周期（pending → in_progress → completed）
- 可持久化： 可存储在Agent state/memory中

3. 在 Agent 中的集成

`TodoListMiddleware` 在使用 `create_deep_agent` 创建智能体时会默认启用，无需额外手动添加。如果开发者需要构建自定义智能体，也可以显式地引入该中间件，并选择性地配置系统提示

词以引导智能体更好地使用该工具。

代码块

```
1  from langchain.agents import create_agent
2  from langchain.agents.middleware import TodoListMiddleware
3
4  # TodoListMiddleware 默认包含在 create_deep_agent 中
5  # 自定义智能体配置示例
6  agent = create_agent(
7      model="claude-sonnet-4-5-20250929",
8      middleware=[
9          TodoListMiddleware(
10             system_prompt="在处理复杂任务时，务必使用 write_todos 工具更新待办清单。"
11         ),
12     ],
13 )
```

这种机制参考了现代 AI 编程辅助工具（如 Claude Code）的设计理念，通过显式的规划步骤显著提高了智能体在长链路任务中的稳定性和逻辑性。

三、task：智能委派系统

1. 核心价值：上下文隔离

`task` 工具的核心价值在于实现“上下文隔离”(Context Quarantine)。主智能体作为“CEO”，其上下文应该保持清爽，只存放最高级别的战略文件和最终决策报告。通过 `task` 工具将复杂、细节繁多的子任务委派给子智能体，确保主智能体的上下文不被污染。

Deep Agents 中的 `task()` 工具是 `SubAgentMiddleware` 的核心接口，旨在通过上下文隔离实现高效的任務分发与委派。

其核心作用是将复杂的任务分解并“下放”给专门的子智能体（Subagent）执行，从而避免主智能体（Supervisor Agent）的上下文窗口被冗长的中间步骤或过多工具调用所干扰。

2. 核心作用与优势

- 上下文隔离：主智能体调用 `task()` 时，该任务在一个独立的子环境中运行。子智能体只处理特定任务，其产生的中间数据、工具调用细节和上下文不会“污染”主智能体的状态。
- 分治策略：主智能体只需关注高层逻辑和任务调度，而无需处理每个子任务的实现细节。这显著降低了处理多步骤、高复杂度任务时的逻辑负担。

- 专项能力提升：开发者可以为子智能体配置特定的工具集、模型甚至自定义系统指令，使其在特定领域（如数据分析、文件处理或外部 API 调用）具备比通用主智能体更高的专业度。

3. 工作原理

当 `SubAgentMiddleware` 被启用后，智能体将获得一个 `task` 工具。其工作流程如下：

- a. 触发委派：主智能体判定某项任务适合由专业子智能体执行时，通过 `task()` 工具发起调用，并传入任务指令。
- b. 环境切换：执行流转入配置好的子智能体环境。该环境可以配置独立的模型、工具和中间件。
- c. 闭环返回：子智能体执行完毕后，将任务的结果摘要返回给主智能体，中间的执行路径、工具调用日志等细节通常会被封装或截断，确保主智能体始终保持简洁的上下文。

4. 代码实现示例

代码块

```
1  from langchain.agents import create_agent
2  from deepagents.middleware.subagents import SubAgentMiddleware
3
4  # 定义一个子智能体配置
5  subagent_config = {
6      "name": "weather_agent",
7      "description": "专用于查询天气的子智能体。",
8      "system_prompt": "使用 get_weather 工具获取信息。",
9      "tools": [get_weather], # 为子智能体绑定特定工具
10     "model": "gpt-4o",
11 }
12
13 # 将中间件注入到智能体中
14 agent = create_agent(
15     model="claude-sonnet-4-5-20250929",
16     middleware=[
17         SubAgentMiddleware(
18             subagents=[subagent_config]
19         )
20     ],
21 )
```

5. 关键点说明

- 通用子智能体：除了用户自定义的子智能体外，每个 Deep Agent 默认还拥有一个 `general-purpose` 子智能体，它继承了主智能体的所有工具和指令，专门用于执行那些需要隔离上下文但无需特殊工具配置的常规任务。

- 图结构兼容：如果简单的配置无法满足需求，你还可以将预先构建好的 `LangGraph` 对象（`create_agent`或者`StateGraph`）作为子智能体封装在 `CompiledSubAgent` 中，实现极其复杂的逻辑编排。

四、设计的think_tool工具

think_tool是研究流程中的一个关键工具，从技术实现的角度看，**可以不需要这个工具，但强烈不建议移除**，原因如下：

为什么从技术上可以移除：

1. **工具本身确实没有复杂代码**：如您所说，它只是接收一个字符串并返回一个确认信息
2. **功能可以通过提示词实现**：理论上，可以通过在提示词中加入"请思考以下问题..."来实现类似功能
3. **不影响核心搜索功能**：移除后不会影响 `zhipu_web_search` 工具的正常运行

为什么强烈不建议移除：

1. 架构设计价值

代码块

```
1  # 当前架构的思考回路：
2  搜索 → think_tool（强制反思） → 决策 → （继续搜索/停止）
3
4  # 如果移除后的可能情况：
5  搜索 → 可能直接继续搜索（无停顿） → 可能过度搜索 → 可能信息冗余
```

2. workflow完整性

- **强制纪律**：工具调用强制研究助手遵守"搜索-反思-决策"的循环
- **可追踪性**：每次调用都产生可追踪的记录
- **状态管理**：帮助研究助手明确自己的研究状态

3. 认知心理学依据

从文档中的描述看，这个工具体现了认知心理学中的"元认知"原理：

- **监控**：评估当前的知识状态
- **调节**：调整后续的学习/研究策略
- **规划**：决定下一步行动

4. 工程实践意义

- **模块化**：将思考过程抽象为独立工具，便于维护和调试
- **可扩展性**：未来可以在此工具中添加更多功能（如：评估信息质量、记录思考日志等）
- **一致性**：确保所有研究任务都遵循相同的思考框架